

LingBot-VLA 2.0 × reBot B601: 采集到上机全链路

基于 rebot_datacollect 采集仓与 pr3967-combined 训练/推理栈的端到端落地流程

采集:lerobot v0.6.1 + rebot_datacollect 训练/推理:lerobot pr3967-combined 平台:A100-80GB + reBot B601

摘要 本文给出 reBot B601 单臂从数据采集到真机推理的完整操作链: rebot_datacollect 闸门式采集 → 生成 robot_config 与 norm_stats → lerobot-train(单卡 B=3,1.99 samples/s)→ 推理开关固化 → lerobot-rollout 上机。两个必须先确认的硬件事实: 本分支自带的 **reBot 驱动是 B601-DM(达妙电机)**, 采集仓驱动是 **B601-RS(RobStride 电机)**, RS 臂上机前需先移植驱动;wrist_depth 深度通道本期不进策略, 只用 front + wrist 两路 RGB。

关键词: reBot B601;LingBot-VLA 2.0; 数据采集;lerobot-train;lerobot-rollout

1 流程总览

Listing 1: 全链路五步

```
rebot_datacollect 采集(闸门式)
-> 生成 robot_config.yaml + norm_stats.json
-> lerobot-train(本分支)
-> bake 推理开关进 ckpt config.json
-> lerobot-rollout 上真机
```

两套环境**不要混**: 采集用 conda data_collect(lerobot v0.6.1 + 三件套插件); 训练与 rollout 用本分支环境 (只有这里有 lingbot_vla_v2 policy 和 rebot_b601_follower)。

1.1 先确认: 你的臂是 -DM 还是 -RS

表 1: 两个 reBot 驱动的分布 (上机前必须核对)

	电机 / 总线	所在位置
rebot_follower	B601-RS,RobStride,SocketCAN/PCAN	采集仓 rebot_datacollect
rebot_b601_follower	B601-DM, 达妙,motorbridge	本分支 (训练/rollout 侧)

采集用哪个驱动都行 (数据集格式与驱动无关); 但 **rollout 必须**在本分支环境跑。DM 臂直接用本文命令;RS 臂需先把采集仓的 RobStride 驱动移植为本分支 robot 插件 (参照 src/lerobot/robots/rebot_b601_follower/ 的结构), 训练不受影响, 可先行。

2 采集 (rebot_datacollect,data_collect 环境)

Listing 2: 采集标准流程 (详见采集仓 scripts/)

```
# 一次性:装环境 + 插件
bash scripts/setup_env.sh
# CAN 起来 + 校准(仅首次)
bash scripts/setup_rebot_can.sh
lerobot-calibrate --robot.type=rebot_follower \
  --robot.id=follower1 --robot.port=can0 \
  --robot.can_adapter=socketcan
# 遥操作 dry-run(rerun 可视化确认映射)
bash scripts/teleop_rebot.sh
# 闸门式录制(每段固定时长,当场保留/丢弃)
PY=~/.miniconda3/envs/data_collect/bin/python CAN=can0 \
  FRONT_CAM=/dev/v4l/by-id/<front-cam-id> \
  REPO_ID=<user>/rebot_pick_place \
  TASK="pick up the black object and place it in the box" \
  EPISODES=50 EP_TIME=15 PUSH=false \
  bash scripts/record_rebot_gated.sh
```

采集仓用命换来的坑,照做:

- Orbbec 腕部相机**必须 USB3**,固件每次上电只允许一个干净会话(脚本带自动 USB reset);掉 USB2 直接挂。
- 相机一律用 /dev/v4l/by-id/ 路径,不用 /dev/videoN(插拔变号)。
- NONBLOCK=1 默认开(循环约 77 Hz,阻塞只有约 30 Hz)。
- STREAM_ENCODE 默认关——开过一次 SIGINT 死锁丢 27 段,别碰。
- 数据集落盘 ~/.cache/huggingface/lerobot/ 下,RESUME=1 可续录;Jetson 上先 maxn_lock.sh 锁频。

采完核对 keys(下一步要对照):observation.state 7 维 (6 关节 + 夹爪)、action 7 维 (与 state 同坐标系)、observation.images.front、observation.images.wrist、observation.images.wrist_depth(uint16 mm,本期不用)。

3 robot_config 与 norm_stats

lingbot_vla_v2 ckpt 用统一 canonical 布局: 关节 arm.position(14 维槽位)、相机 camera_top/camera_wrist_left/camera_wrist_right。robot_config 把 reBot 数据集原始 key 映射上去;缺位 canonical 相机自动 mask (与推理行为一致),两路物理相机只映射两条。

Listing 3: rebot_b601_robot_config.yaml 全文

```
states:
  - observation.state.arm.position:
      origin_keys:
        - observation.state:
            start: 0
            end: 7          # 7 维进 14 维槽位,其余补零
actions:
```

```

- action.arm.position:
  origin_keys:
    - action:
      start: 0
      end: 7
  subtract_state: false
images:
- observation.images.camera_top:
  origin_keys: observation.images.front
- observation.images.camera_wrist_left:
  origin_keys: observation.images.wrist
# camera_wrist_right 不映射 -> 自动 mask;有第三路再加
norm_stats: /home/<you>/rebot_lingbot/rebot_b601_norm_stats.json

```

训练日志出现 camera_wrist_right left unfilled 的 warning 是预期行为, 不是错误。
norm_stats 由数据集全量帧统计生成 (本分支环境跑, root 换成实际落盘路径):

Listing 4: 生成 rebot_b601_norm_stats.json

```

import json
import numpy as np
from lerobot.datasets.lerobot_dataset import LeRobotDataset

ds = LeRobotDataset("<user>/rebot_pick_place",
                    root="<实际落盘路径>")
s = np.asarray(ds.hf_dataset["observation.state"],
               dtype=np.float64)
a = np.asarray(ds.hf_dataset["action"], dtype=np.float64)
assert s.shape[1] == 7 and a.shape[1] == 7

def stats(x):
    return {"min": x.min(0).tolist(),
            "max": x.max(0).tolist(),
            "mean": x.mean(0).tolist(),
            "std": (x.std(0) + 1e-8).tolist()} # 防夹爪常量除零

out = {"norm_stats": {
    "observation.state.arm.position": stats(s),
    "action.arm.position": stats(a)}}
json.dump(out, open("rebot_b601_norm_stats.json", "w"),
          indent=2)

```

4 训练 (本分支环境)

Listing 5: 单卡 80GB; 显存更小就降 batch_size

```

lerobot-train \
--dataset.repo_id=<user>/rebot_pick_place \
--dataset.root=<实际落盘路径> \
--policy.type=lingbot_vla_v2 \

```

```
--policy.pretrained_path=~/.lvla_scratch/robotwin-6b-lerobot \
--policy.robot_config_path=~/.rebot_lingbot/rebot_b601_robot_config.yaml \
--policy.norm_stats_path=~/.rebot_lingbot/rebot_b601_norm_stats.json \
--policy.dtype=bfloat16 \
--policy.optimizer_fused=true \
--policy.push_to_hub=false \
--batch_size=3 --steps=30000 --log_freq=50 --num_workers=4 \
--save_checkpoint=true --save_freq=5000 \
--output_dir=~/.output_learner_train/rebot/lingbot_v2_A \
--job_name=rebot_lingbot_v2_A --wandb.enable=false
```

多卡 (命令其余部分同上, 只改启动方式与 batch):

```
accelerate launch --num_processes=4 $(which learner_train) ... --batch_size=2
```

batch_size 是每卡语义, DDP 每卡上限 2, 3 会 OOM。

纪律 (全部来自实测): robot_config_path / norm_stats_path 必须显式传 (robotwin ckpt 内嵌的是 robotwin 相机名); 不要设 sdpa_backend (CUDA 训练 NaN、EFFICIENT 无可用 kernel, 见开关验证报告); $B \leq 4$ 不要开 gradient_checkpointing (+38% 慢)。吞吐锚点: 单卡 $B=3$ 约 1.99 samples/s, 30k 步约 12.6 h; 4 卡 $B=2$ /卡约 8.0 samples/s, 约 8.3 h。

5 推理开关固化与上机 rollout

训练出的 ckpt 默认不带推理优化开关, 先一次性固化 (之后 rollout 只需 --policy.path):

Listing 6: 固化五开关进 ckpt 的 config.json

```
import json, pathlib
p = (pathlib.Path.home() /
     "output_learner_train/rebot/lingbot_v2_A"
     "/checkpoints/last/pretrained_model/config.json")
c = json.loads(p.read_text())
c.update({"compile_predict_velocity": True,
         "compile_predict_velocity_mode":
             "max-autotune-no-cudagraphs",
         "compile_prefix": True,
         "preprocess_device": "cuda",
         "num_steps": 10})
p.write_text(json.dumps(c, indent=2))
```

固化路径已被启动验证矩阵 I6 臂端到端复刻通过 (稳态 258 ms, 动作有限)。首次推理有一次性编译 (几分钟); ckpt 拷到随臂机器后编译缓存会重建一次。

Listing 7: rollout 上机 (reBot B601-DM)

```
lerobot-rollout \
--policy.path=~/.output_learner_train/rebot/lingbot_v2_A/checkpoints/last/
pretrained_model \
--robot.type=rebot_b601_follower \
--robot.port=can0 --robot.can_adapter=socketcan \
--robot.id=follower1 \
```

```
--robot.cameras="{ front: {type: opencv, index_or_path: /dev/v4l/by-id/<front-cam
-id>, width: 640, height: 480, fps: 30, fourcc: MJPG}, wrist: {type: opencv,
index_or_path: /dev/v4l/by-id/<orbbec-id>, width: 640, height: 480, fps: 30,
fourcc: MJPG}}" \
--strategy.type=base --inference.type=sync \
--task="pick up the black object and place it in the box" \
--fps=30 --duration=60 --display_data=false --play_sounds=false
```

要点:

- 相机 key 必须叫 **front / wrist**, 与 robot_config 的 origin_keys 一字不差。
- 达妙串口桥用 `--robot.port=/dev/ttyACM0`(默认 `can_adapter=damiao`); `SocketCAN/P-` `CAN` 用 `can0` + `--robot.can_adapter=socketcan`。
- **task 文本必须和采集时的 TASK 一致** (语言条件进模型)。
- 第一次上机先 `--duration=10` 短跑, 确认动作方向与归一化正确再上长任务。
- Orbbec 在 rollout 侧当普通 UVC 相机用 (只取 RGB)。

要录 eval 数据, 追加三个参数:

- `--dataset.repo_id=<user>/eval_rebot_lingbot_v2`
- `--dataset.single_task="< 与 TASK 一致 >"`
- `--dataset.push_to_hub=false`

6 边界与后续项

- `wrist_depth` 暂未进策略:`use_depth_align` 通路存在但未在 reBot 数据上验证;v1 只用 RGB。
- 两路相机训出的模型 `camera_wrist_right` 全程 mask, 训练推理行为一致; 日后加第三路相机需重新采集含该视角的数据再训 (`mask ≠` 模型会用该视角)。
- B601-RS 臂的 rollout 驱动移植是独立工作项 (§1.1)。
- 50 段 × 15 s 规模的数据集, 30k 步已多个 epoch; 可降 steps, 5k 一存看 eval 曲线再定。

参考与产物

[1] 采集仓: github.com/miracle-techlink/rebot_datacollect(Apache-2.0)。

[2] 通用命令手册:

`docs/accel/lingbot_vla_train_rollout_commands.md`;

开关验证报告:`docs/accel/lingbot_vla_switch_table.pdf`。

[3] 本分支 reBot 驱动:

`src/lerobot/robots/rebot_b601_follower/(含双臂 bi_rebot_b601_follower)`。

[4] 吞吐与时延锚点:A100 实测, 宿主机有争抢, 见 `docs/accel/ 优化总报告`。